Decroly
SANTANDER

UD8

LENGUAJE DE PROGRAMACIÓN JAVA

8.1 Acceso a datos JDBC

MP_0485
Programación

Introducción

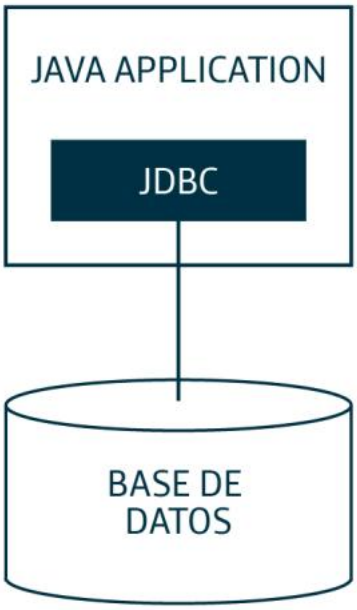
En esta unidad perseguimos los siguientes objetivos:

- Conocer las librerías de clases Java relacionadas con el acceso a bases de datos.
- Establecer conexión con una base de datos utilizando el API JDBC.
- Conocer las clases principales de la librería `java.sql`.

El API JDBC

El API JDBC provee un mecanismo sencillo para acceder a bases de datos relacionales desde aplicaciones Java. Se encuentra disponible en el paquete `java.sql`, que pertenece a la "JRE System Library", por lo que no es necesario cargar ninguna librería externa

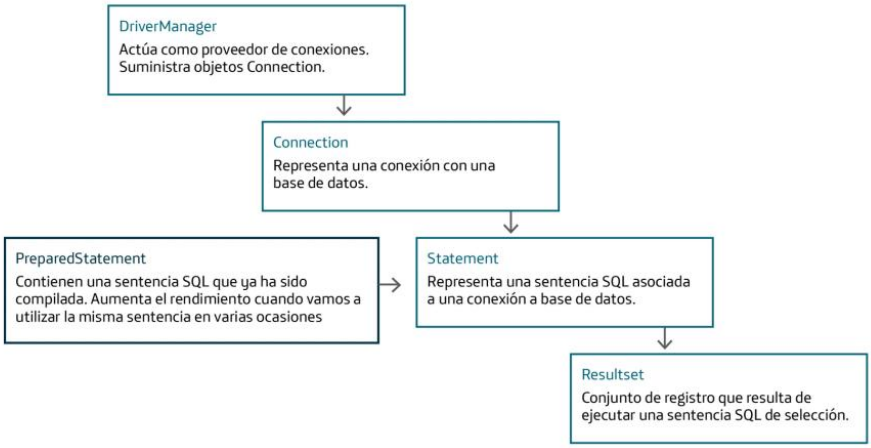
Sin embargo, sí requiere incluir las sentencias `import` necesarias para importar las clases que vayamos a utilizar. La principal característica de JDBC es que permite independizar nuestro código Java de las peculiaridades de cada DBMS (Data Base Management System), de modo que nuestro programa Java pueda funcionar exactamente igual accediendo a una base de datos MySQL, una base de datos SQL Server, Oracle, etc..



JDBC se sitúa como una capa dentro de nuestra aplicación Java, capaz de comunicarse con el DBMS. En función del DBMS que soporte la base de datos, JDBC necesitará un **driver o conector** especial para Java que habrá que añadir al proyecto. Para los ejemplos de esta lección nos basaremos en una base de datos MySQL. Desde nuestro programa Java realizamos llamadas estándar a métodos de las clases del API JDBC, y es el driver o conector el que se encarga de traducir estas llamadas a un lenguaje que entienda el DBMS.

Clases más importantes del API JDBC

En este apartado veremos las clases más importantes del API JDBC.



Esquema de clases del API JDBC.

- **DriverManager**: como hemos comentado anteriormente, dentro del proyecto es necesario incluir un driver o conector por cada DBMS con el que nos queramos comunicar. Pues bien, será la clase DriverManager la que se encargue de gestionar estos drivers y de proveer al programa de las bases de datos que necesite.

- **Connection**: representa una conexión con una base de datos concreta, que será previamente suministrada por el DriverManager. La conexión con la base de datos se realiza a través de una cadena de conexión, que contendrá los datos de conexión necesarios.

- **Statement**: representa una sentencia SQL, que podrá ser de selección de datos (SELECT), de actualización (INSERT, DELETE, UPDATE) o de de nición (CREATE, ALTER, DROP).

- **PreparedStatement**: en ocasiones tenemos que ejecutar muchas sentencias SQL que son iguales, pero con distintos datos o parámetros. Por ejemplo: si tenemos que añadir 200 artículos en una tabla necesitaremos ejecutar 200 sentencias INSERT exactamente iguales, donde sólo varían los datos de cada artículo. La ejecución de cada uno de estos INSERT supone para el DBMS realizar las siguientes tareas:
    - Comprobar que la sentencia SQL es correcta.
    - Convertir los datos al tipo adecuado para el DBMS.
    - Ejecutar la sentencia SQL.

  Realizar estas tareas 200 veces podría suponer un tiempo considerable. Para hacer más eficiente este proceso PreparedStatement nos brinda un sistema para precompilar la sentencia SQL y guardarla para ser ejecutada inmediatamente, sin necesidad de analizarla en cada caso. PreparedStatement también nos permite añadir seguridad a nuestros programas, ya que podemos evitar "inyecciones SQL". Pulsa aquí para conocer lo que son las inyecciones SQL.

- **ResultSet**: los objetos ResultSet se crean como resultado de la ejecución de una sentencia SELECT y contienen el conjunto de registros resultado de su ejecución

Establecer conexión con la base de datos

En este apartado vamos a establecer conexión con una base de datos MySQL llamada `dcuniverse`. Todos los ejemplos de código Java que aparecerán en esta unidad se conectarán a una base de datos MySQL, situada en un nuestro propio equipo.

Todos los ejemplos trabajan contra la base de datos `dcuniverse` y con una tabla llamada `characters`, que tiene las siguientes columnas o campos:

Column	Type	Default Value	Nullable
id	bigintunsigned		NO
heroName	varchar(50)		NO
fullName	varchar(100)		YES
image1	text		YES
image2	text		YES
image3	text		YES
gender	varchar(20)		YES
race	varchar(50)		YES
alignment	varchar(10)		YES

Para lograr conectar a la base de datos `dcuniverse` utilizaremos las siguientes clases:

- DriverManager: contiene información de todos los drivers o conectores cargados en memoria y actúa como proveedor, suministrando conexiones siempre y cuando tenga acceso a los drivers necesarios. Nosotros ya le hemos preparado el camino al DriverManager para que encuentre dichos drivers en el apartado anterior.

- Connection: representa una conexión con una base de datos determinada y es suministrada por el DriverManager.

Para establecer conexión con una base de datos se utiliza el método estático getConnection() de la clase DriverManager, que retorna un objeto Connection. El método getConnection() tiene el siguiente formato:

```
Connection nombreObj = DriverManager.getConnection(String cadenaConexion,
String usuario, String contraseña);
```

El primer argumento es la cadena de conexión, que contendrá información sobre la base de datos a la que deseamos acceder. Los argumentos segundo y tercero contienen el nombre de usuario y contraseña del usuario autorizado a utilizar la base de datos.

La cadena de conexión

Una cadena de conexión (connection string) es un String que contiene información de acceso a una base de datos. El formato utilizado es diferente en función del proveedor de bases de datos. JDBC nos permite utilizar el mismo código para acceder a un gran número de formatos de bases de datos cambiando solamente la cadena de conexión.

Puesto que los elementos y el formato de la cadena de conexión varían en función del proveedor de bases de datos, es difícil explicarlo de forma específica, pero, en términos generales, toda cadena de conexión se divide en las siguientes áreas:

- Proveedor: hace referencia al gestor de base de datos utilizado (jdbc:sqlserver, jdbc:derby, jdbc:mysql, etc.).

- Dirección IP o Nombre de servidor: se refiere al servidor donde está alojada la base de datos. Se utiliza en SGBD tales como SQL Server, Oracle o MySQL, que trabajan con tecnología cliente/servidor. Puede ser una DNS o una dirección IP.
- Nombre de la instancia: en SQL Server o MySQL un servidor puede contener varias instancias, cada instancia con sus propias bases de datos.
- Número de puerto: necesario para establecer la conexión con el servidor. En MySQL es habitual que sea el 3306.
- Nombre de la base de datos: nombre de la base de datos a la que queremos acceder en nuestro programa.

Vamos a generar una clase de gestión de la conexión a la base de datos en una clase estática para que la conexión se única. Podíamos generar un patrón de Singleton, pero lo veremos más adelante en el curso.

```java
public class SQLDataBaseManager {

    private static final String DRIVER = "com.mysql.cj.jdbc.Driver";
    private static final String URL = "jdbc:mysql://localhost:3306/";
    private static final String SCHEMA = "dcuniverse";
    private static final String USUARIO = "developer";
    private static final String CLAVE = "daw02";

    public static Connection getConnection(){
        Connection connection = null;
        try {
            Class.forName(DRIVER);
            connection = DriverManager.getConnection(URL+SCHEMA, USUARIO, CLAVE);

        } catch (ClassNotFoundException e) {
            System.out.println("Error de acceso al driver: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("Error de SQL: " + e.getMessage());
        }
        return connection;
    }
}
```

Hemos parametrizado los valores de la conexión para que sea muy sencillo personalizar la clase para cualquier aplicación y conexión a la BBDD.

Obtener datos de la BBDD

En este apartado vamos a crear una clase que nos sirva para el acceso a datos de nuestro esquema de la base de datos. Haremos uso de la clase anterior (`SQLDataBaseManager`) donde obteníamos la conexión a la BBDD.

```
public class SQLAccessDCuniverse {

    public List<String> getCharactersName(){
        List<String> charactersNames = new LinkedList<>();

        //Setencia SQL
        String getChNames = "SELECT heroName FROM characters";

        try (Connection connection = SQLDataBaseManager.getConnection();
             Statement statement = connection.createStatement();
        ResultSet dataSet = statement.executeQuery(getChNames);) {
            while(dataSet.next()){
                charactersNames.add(dataSet.getNString(1));
            }

        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
        return charactersNames;
    }
```

En este primer método vamos a obtener solo los nombre de los `Characters` y lo guardaremos en una `List<String>`. Haciendo uso de un `try-catch` con recursos obtendremos la conexión y la sentencia que deseamos ejecutar en la BBDD y su ejecución cuyo resultado se guardará en un objeto `ResultSet`.

Posteriormente recorreremos ese objeto ResultSet, accediendo a las columnas y guardando los resultados nuestra `List<String>`.

Vamos a crear ahora un nuevo método para obtener todos los `Characters` que hay en la BBDD. El esquema es muy parecido al del método anterior.

```java
public List<CharacterDC> getCharacters(){
    List<CharacterDC> charactersNames = new LinkedList<>();
    //Setencia SQL
    String getChNames = "SELECT * FROM characters";
    try (Connection connection = SQLDataBaseManager.getConnection(); Statement statement = connection.createStatement();
            ResultSet dataSet = statement.executeQuery(getChNames);) {
        while(dataSet.next()){
            int id = dataSet.getInt(1);
            String name = dataSet.getNString(2);
            String fullname = dataSet.getNString(3);
            String gender = dataSet.getNString(7);
            String race = dataSet.getNString(8);
            String aligment = dataSet.getNString(9);

            CharacterDC c1 =  new CharacterDC(id, name, fullname, gender, race, aligment);
            charactersNames.add(c1);
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
    return charactersNames;
}
```

En este caso el ResultSet tiene más columnas (todas las de la tabla Characters) y debemos guardar cada uno de ellos una variable.

Pero que ocurre si nuestra consulta tiene un parámetro de búsqueda, un ID, un nombre en particular, etc. En estos casos haremos uso de PreparedStament, el cual nos permite introducir parámetros dinámicos en nuestras consultas. Vamos a crear un nuevo método en la clase.

```java
public List<CharacterDC> getCharacterByName(String characterName){
    List<CharacterDC> charactersNames = new LinkedList<>();
    String sqlStatement = "SELECT * FROM characters where heroName = ?";
    try (Connection connection = SQLDataBaseManager.getConnection();
         PreparedStatement statement = connection.prepareStatement(sqlStatement);) {
        statement.setString(1, characterName);
        ResultSet dataSet = statement.executeQuery();
        while(dataSet.next()){
            int id = dataSet.getInt(1);
            String name = dataSet.getNString(2);
            String fullname = dataSet.getNString(3);
            String gender = dataSet.getNString(7);
            String race = dataSet.getNString(8);
            String aligment = dataSet.getNString(9);

            CharacterDC c1 =  new CharacterDC(id, name, fullname, gender, race, aligment);
            charactersNames.add(c1);
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
    return charactersNames;}
```

En la variable de tipo `String` `sqlStatement` introducimos un carácter "?" donde debería ir el parámetro. En vez de usar un objeto del tipo `Statement` tenemos que usar un objeto `PreparedStatement` e ir añadiendo los parámetros a la sentencia mediante la instrucción:

```
statement.setString(1, characterName);
```

Y ejecutar la consulta:

```
ResultSet dataSet = statement.executeQuery();
```

En este ejemplo realizamos una consulta con dos parámetros:

```
public List<CharacterDC> getCharacterByNameAndAlterEgo(String characterName, String alterEgo){
    List<CharacterDC> charactersNames = new LinkedList<>();
    String sqlStatement = "SELECT * FROM characters where heroName = ? and fullName = ?";
    try (Connection connection = SQLDataBaseManager.getConnection();
         PreparedStatement statement = connection.prepareStatement(sqlStatement);) {
        statement.setString(1, characterName);
        statement.setString(2, alterEgo);
        ResultSet dataSet = statement.executeQuery();
        while(dataSet.next()){
            int id = dataSet.getInt(1);
            String name = dataSet.getNString(2);
            String fullname = dataSet.getNString(3);
            String gender = dataSet.getNString(7);
            String race = dataSet.getNString(8);
            String aligment = dataSet.getNString(9);
            CharacterDC c1 =  new CharacterDC(id, name, fullname, gender, race, aligment);
            charactersNames.add(c1);
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
    return charactersNames;}
```

Actualización, eliminación e inserción de datos

Normalmente en nuestras aplicaciones tenemos que insertar o editar registros, así como eliminarlos. Para ello vamos a añadir tres métodos en nuestra clase que permiten la realización de estas acciones.

En nuestro modelo de datos, tenemos una relación entre la tabla `Characters` y `PowerStats`, de tal forma que cada registro de `PowerStats` tiene una vinculación con `Characters`. De tal forma

que cuando queremos eliminar un registro de la tabla `Characters` debemos de eliminar también su registro asociado en `PowerStats`.

En este caso volvemos a hacer uso de `PreparedStatement`, pasando como parámetro el id de cada tabla.

```
public int deleteById(int id){
    int elements = -1;

    String sqlStatement = "DELETE FROM powerstats where characters_id = ?";
    String sqlStatement2 = "DELETE FROM characters where id = ? ;";

    try (Connection connection = SQLDataBaseManager.getConnection();
         PreparedStatement statement1 = connection.prepareStatement(sqlStatement);
         PreparedStatement statement2 = connection.prepareStatement(sqlStatement2);) {

        statement1.setInt(1, id);
        statement2.setInt(1, id);

        elements = statement1.executeUpdate();
        elements += statement2.executeUpdate();

    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
    return elements;
}
```

De una forma análoga podemos realizar la inserción de un registro de la tabla `Characters`, en este caso no realizamos la inserción en la tabla `PowerStats`.

```java
public int insertCharacter(CharacterDC character){
    int response = -1;
    String sqlStatement = "INSERT INTO characters (heroName, fullName, image1, image2, image3, gender, race, alignment)"
            + "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
    try (Connection connection = SQLDataBaseManager.getConnection();
         PreparedStatement statement = connection.prepareStatement(sqlStatement);) {

        statement.setNString(1, character.getHeroName());
        statement.setNString(2, character.getFullName());
        statement.setNString(3, character.getImage1());
        statement.setNString(4, character.getImage2());
        statement.setNString(5, character.getImage3());
        statement.setNString(6, character.getGender());
        statement.setNString(7, character.getRace());
        statement.setNString(8, character.getAligment());
        response = statement.executeUpdate();
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
    return response;
}
```

Finalmente creamos otro método para actualizar un registro ya existente.

```java
public int updateCharacterById(CharacterDC character){
    int response = -1;
    String sqlStatement = "UPDATE characters set heroName = ? , fullName = ?, alignment = ? where id = ?";

    try (Connection connection = SQLDataBaseManager.getConnection();
         PreparedStatement statement = connection.prepareStatement(sqlStatement);) {

        statement.setNString(1, character.getHeroName());
        statement.setNString(2, character.getFullName());
        statement.setNString(3, character.getAligment());
        statement.setInt(4, character.getId());

        statement.executeUpdate();

    } catch (Exception e) {
        System.out.println("ERROR: "+e.getMessage());        }
    return response;
}
```

Referencias

Apuntes elaborados a partir de la siguiente documentación:

[1] Apuntes Fernando Barber y Ricardo Ferris. Universidad de Valencia.

[2] Apuntes Programación de José Antonio Díaz-Alejo. IES Camp de Morvedre.

[3] Apuntes de Programación de Carlos Cacho y Raquel Torres. Ceedcv.

[4] Apuntes de Programación Edix Digital Workers.

Licencia

CC BY-NC-SA 3.0 ES **Reconocimiento – No Comercial – Compartir Igual (by-nc-sa)**
No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.
NOTA: Esta es una obra derivada de la original realizada por Carlos Cacho y Raquel Torres.